React, Behind the Scenes

React, Behind the Scenes — A Production Field Guide

How React actually decides what to render, why hooks are a linked list on a fiber, every hook with production-shaped examples, why `useEffect` is overused (and what to reach for instead), and a 10-project build ladder.

Table of Contents

Part I — The Engine

The core idea: UI is a function of state

Before Fiber, before hooks, before any of the machinery — React rests on one idea: **UI = f(state)**. You describe what the UI should look like for a given state, and React's job is figuring out the cheapest way to get the real DOM from its current shape to that description. Everything else in this guide is the implementation of that one sentence.

Two consequences fall out of it immediately:

- You never mutate the DOM directly — you mutate *state*, and describe the result of the new state.
- React must have some way to compare "what the UI should look like now" against "what it looked like last time," without touching the real, slow DOM to do the comparison.

That second point is the entire reason the Virtual DOM exists.

The Virtual DOM: a plan, not a copy

A common misconception: the Virtual DOM is *not* "a faster copy of the DOM." It's a plain JavaScript object tree — the output of your components calling `createElement` (which JSX compiles to). Each node looks roughly like:

```
const element = {
  type: 'button',
  props: {
    className: 'btn-primary',
    onClick: handleClick,
    children: 'Save'
  }
}
```

Building this object tree is cheap — it's just function calls and object literals, no layout, no paint. React builds a new one on every render and compares it against the tree from the previous render. That comparison — **reconciliation** — is what decides the minimal set of real DOM mutations needed.

> **Why this matters.** Direct DOM writes are expensive because they can trigger layout recalculation and repaint. Diffing two JS objects in memory costs microseconds. React trades a small amount of CPU work for a large reduction in DOM writes — that trade only pays off because JS object diffing is so much cheaper than the DOM operations it avoids.

Babel and JSX: what the browser actually runs

`<button onClick={handleClick}>Save</button>` is not valid JavaScript — no browser can execute it. JSX is syntax sugar that exists purely for authoring convenience, and it never ships to the browser as written. **Babel** (specifically `@babel/preset-react`, bundled into every framework's build tool — Vite, Next.js, Create React App) transforms it into the plain function calls from the previous section, at build time, before any code reaches a browser or bundler.

```
// What you write
const element = <button onClick={handleClick}>Save</button>;
```

```
// What Babel emits — the "classic" runtime, React ≤16 default
const element = React.createElement('button', { onClick: handleClick },
          'Save');
```

```
// What Babel emits — the "automatic" runtime, React 17+ default
import { jsx as _jsx } from 'react/jsx-runtime';
const element = _jsx('button', { onClick: handleClick, children: 'Save' });
```

> **Why the runtime changed in React 17.** The classic output calls `React.createElement`, which meant every single file using JSX needed `import React from 'react'` in scope even if the code never referenced `React` directly — purely so that implicit call could resolve. The automatic runtime has Babel import `jsx` / `jsxs` directly from `react/jsx-runtime` and call those instead, so the `import React from 'react'` boilerplate at the top of every component file became unnecessary.

This is the concrete reason "components are just functions" is true rather than a simplification: after Babel runs, a component is a plain function that returns a plain object (or a tree of them) built from ordinary function calls — `<Row key={item.id} data={item} />` and `React.createElement(Row, { key: item.id, data: item })` are the same code before and after a build step, not two different things.

Babel's JSX transform is a separate concern from `@babel/preset-env`, which is what downlevels modern JS syntax (optional chaining, nullish coalescing, newer array methods) to whatever browser targets your project declares support for — a JSX-only codebase with no legacy browser targets can skip `preset-env` entirely and let the bundler handle syntax as-is; most real projects run both presets together as one build step.

Reconciliation: how the diff actually works

Naive tree diffing is `O(n³)` — comparing every node in one tree against every node in another. React makes it `O(n)` with two heuristics that are deliberate trade-offs, not accidents:

1. **Different element types produce different trees.** If a `<div>` becomes a `<span>` between renders, React doesn't try to diff their children — it tears down the old subtree entirely (unmounting, losing state) and builds a new one from scratch.
2. **Keys tell React which children are "the same" across renders.** Without keys, list diffing falls back to positional comparison — index 0 vs index 0, index 1 vs index 1 — which is wrong the instant you insert, remove, or reorder.

> **The classic bug.** Using array `index` as a key. Insert an item at the front of a list keyed by index, and every existing item's key now points at a different logical item — React reuses DOM nodes (and any internal state like text-input contents) for the wrong rows. Use a stable, unique ID from your data. Only fall back to index when the list is truly static and never reordered.

```
// Bad — index shifts whenever the list is reordered/filtered
items.map((item, i) => <Row key={i} data={item} />)

// Good — identity survives reordering
items.map((item) => <Row key={item.id} data={item} />)
```

The list-diffing algorithm, step by step

For a single child, diffing is trivial — same type at the same position, update it; different type, replace it. Lists are the interesting case, and React's actual algorithm (`reconcileChildrenArray` in the source) runs in two passes to stay `O(n)` instead of comparing every old child against every new one:

Pass 1 — walk both lists together, front to back, while positions and keys still match. For each index, if the old child at that index and the new child at that index have the same key (or both have no key) and the same type, React updates it in place and moves on. The instant a key mismatch is hit, pass 1 stops — everything from here needs the slower path.

Pass 2 — index the remaining old children by key into a map, then walk the remaining new children once, looking each one up in that map:

```
// Roughly what happens once pass 1 hits a mismatch
const existingChildrenByKey = new Map();
oldChildren.forEach((child) => existingChildrenByKey.set(child.key ??
        child.index, child));

let lastPlacedIndex = 0;
for (const newChild of remainingNewChildren) {
  const match = existingChildrenByKey.get(newChild.key);
  if (match && match.type === newChild.type) {
    // found the same logical item — reuse its fiber (and its DOM node, its
        state)
    if (match.oldIndex < lastPlacedIndex) {
      markForPlacement(match); // it needs to physically move in the DOM
    } else {
      lastPlacedIndex = match.oldIndex; // stayed in relative order, no DOM
        move needed
    }
    existingChildrenByKey.delete(newChild.key);
  } else {
    markForPlacement(newChild); // no match — brand-new fiber, new DOM node,
        mounts fresh
  }
}
// anything left in the map had no matching new child — mark for deletion
        (unmount)
existingChildrenByKey.forEach(markForDeletion);
```

`lastPlacedIndex` is the mechanism behind "does this item need to physically move." If a matched item's original index is *behind* the highest index already placed, it fell out of order relative to items already positioned, and React issues a DOM move for it; if not, it's still in relative order and can stay untouched. This is also exactly why keyed reordering preserves component state (input focus, scroll position, internal `useState`) for moved items but a type or key mismatch does not — a mismatch never reaches the "reuse the fiber" branch at all.

Reconciliation decides *what changed*. It does not decide *when* or *in what order* React gets to do the work — that's Fiber's job.

Fiber: turning render into interruptible units of work

Before React 16, rendering was a single, synchronous, recursive walk down the element tree — *stack reconciliation*. Once it started, it couldn't stop until it finished the whole tree, even if that meant blocking the main thread for 200ms and dropping frames on an animation.

Fiber replaces the call stack with a data structure you can pause, resume, abort, and prioritize. A **fiber** is a plain JS object representing one unit of work — roughly one component instance — with pointers to its child, its sibling, and its return (parent):

```
const fiberNode = {
  type: MyComponent,
  key: null,
  stateNode: /* the component instance or DOM node */,
  child: /* first child fiber */,
  sibling: /* next sibling fiber */,
  return: /* parent fiber */,
  memoizedState: /* linked list of hooks — see Part II */,
  memoizedProps: /* props from the last completed render */,
  pendingProps: /* incoming props for this render */,
  alternate: /* the fiber from the other tree — see below */,
  effectTag: /* Placement | Update | Deletion */,
}
```

Because it's just an object graph instead of a call stack, React's work loop can walk it with plain iteration and yield back to the browser between units of work:

```
function workLoop(deadline) {
  while (nextUnitOfWork && deadline.timeRemaining() > 0) {
    nextUnitOfWork = performUnitOfWork(nextUnitOfWork);
  }
  // ran out of time this frame — yield to the browser,
  // paint/handle input, then resume later
  requestIdleCallback(workLoop);
}
```

The double-buffered tree. React keeps *two* fiber trees in memory: **current** (what's on screen) and **work-in-progress** (what's being built for the next render). Each fiber's `alternate` pointer links it to its counterpart in the other tree. When work-in-progress finishes and commits, the two trees swap roles — no tree is ever rebuilt from nothing, and there's always a complete, consistent tree to fall back to if work is abandoned mid-flight.

> **Why this matters.** Double buffering is the same technique graphics programming uses to avoid tearing. It's what makes it safe for React to *abandon* an in-progress render (say, a higher-priority update arrives) without ever showing the user a half-built screen.

Render phase vs. commit phase

Every update runs through two distinct phases with very different rules:

	Render phase — figure out what changed	**Commit phase — apply it to the real DOM**
Does	Calls your component functions; builds the work-in-progress fiber tree; diffs against `current`	Applies insertions/updates/deletions to the DOM; runs `useLayoutEffect` synchronously; browser paints; runs `useEffect` callbacks after paint
Interruptible?	Yes — can be thrown away entirely	No — synchronous, never interrupted
Must be pure	Yes — no DOM writes, no side effects	N/A — this is where side effects belong

> **Why the render phase must be pure.** React may call your component function, throw the result away (a higher-priority update preempted it), and call it again — possibly more than once per commit in Strict Mode, specifically to surface this bug in development. If your component body mutates a module-level variable, fires a network request, or writes to a ref as a side effect, you'll get double effects or corrupted state that only shows up under concurrent rendering. Side effects belong in event handlers or `useEffect` / `useLayoutEffect`, which only run during the commit phase and are guaranteed to run exactly once per real commit.

Concurrent rendering: priority, not just speed

Fiber's interruptibility enables React 18's concurrent features. React can now assign different **lanes** (priority levels) to different updates. A user typing into an input is urgent; a large list re-filtering in the background is not. React can start the low-priority render, pause it when the urgent keystroke update arrives, finish the keystroke update first, then resume or restart the background one.

`useTransition` and `useDeferredValue` (covered in Part II) are the two hooks that let you opt specific updates into this lower-priority lane yourself, instead of everything defaulting to the same urgency.

Automatic batching: why one click can cause one render, not three

React 18 batches **every** state update inside a given "tick" of work — event handlers, but also promises, `setTimeout`, and native event listeners (pre-18, only React's own synthetic event handlers batched; a `setTimeout` callback flushed each `setState` separately). Batching means several `setState` calls collapse into a single re-render instead of one render per call.

```
function handleClick() {
  setCount(c => c + 1);   // does NOT re-render yet
  setFlag(f => !f);       // does NOT re-render yet
  setName('updated');     // still hasn't re-rendered
  // React commits ONE render here, with all three updates applied
}
```

```
// Before React 18, this caused THREE renders — now it's one, same as the
         click handler above
setTimeout(() => {
  setCount(c => c + 1);
  setFlag(f => !f);
  setName('updated');
}, 1000);
```

> **Why this matters.** Batching is why `console.log(count)` immediately after `setCount(count + 1)` still prints the old value — the state variable in that closure was never going to update until the next render regardless of batching, but batching is *also* why triggering five state updates from one user action costs one trip through the render/commit pipeline instead of five. If you ever need to force a synchronous, unbatched update (rare — mostly measuring layout mid-handler), `flushSync` from `react-dom` opts a specific update out of batching.

StrictMode's double-invoking, explained

In development, `<StrictMode>` intentionally renders every component function twice, and calls every `useState` initializer, every reducer, and every effect setup + cleanup twice on mount. This is not a bug and it does not happen in production builds — it exists purely to surface impurity while it's still cheap to fix.

```
function Widget() {
  console.log('rendering'); // logs twice per actual commit, in dev, under
        StrictMode
  useEffect(() => {
    console.log('effect ran');
    return () => console.log('cleanup ran');
  }, []);
  // dev/StrictMode order: "effect ran" → "cleanup ran" → "effect ran"
  // production order:      "effect ran"  (once)
}
```

> **The lesson, not the workaround.** If the double-invoke breaks something (a duplicate API call, a duplicate WebSocket connection), the fix is never to remove StrictMode — it's to make the effect properly idempotent, because concurrent rendering can cause the exact same double-invocation in production for unrelated reasons (an abandoned and restarted render). StrictMode is a smoke detector, not the fire.

The Scheduler and priority lanes, one level deeper

React's Scheduler package assigns each unit of work a priority derived from what triggered it:

Trigger	Lane	Behavior
Discrete user input (click, keydown, typing)	Sync / Input-blocking	Rendered immediately, blocks nothing else from being scheduled first
Continuous input (scroll, drag), animations	Default	High priority, but yieldable
`startTransition` / `useDeferredValue` updates	Transition	Low priority — interruptible by anything above it, can be starved indefinitely under continuous high-priority input
Data fetched via Suspense while already showing content	Retry	Scheduled to avoid tearing down visible content until the new data is ready

This table is the mechanical reason the guidance in Part II holds: an urgent keystroke update always wins over a transition-wrapped update, and a transition can sit "pending" for a while if the user keeps typing — which is exactly what `isPending` from `useTransition` is telling you.

Part II — Hooks

How hooks actually work (the part that explains all the “rules”)

Hooks aren’t magic and they aren’t tied to variable names — they’re entries in a **linked list stored on the fiber** (`fiber.memoizedState`), and each hook is identified purely by **call order**, not by name.

```
// Roughly what one hook's node looks like internally
{
  memoizedState: /* the actual state value, or effect deps, etc. */,
  next: /* pointer to the next hook called in this component */,
}
```

On the first render, each `useState` / `useEffect` /etc. call appends a new node to this list. On every re-render, React walks the *same* list in the *same* order and hands each hook call the node at that position. There is no name-based lookup — `useState` call #3 always gets list node #3.

> **This is why the Rules of Hooks exist.** “Don’t call hooks inside conditionals or loops” isn’t a style preference — it’s a consequence of this data structure. If render A calls 4 hooks and render B (same component) calls 3 because an `if` skipped one, every hook after the skipped one now reads the *wrong node* from the linked list. You get state from the wrong hook silently, or a “rendered more hooks than during the previous render” crash if React can detect the mismatch.

This single mechanism explains a lot of hook behavior that otherwise looks arbitrary: why hook order must be static, why custom hooks can freely call other hooks (they’re just composing more list entries into the same fiber), and why each component instance gets its own independent list — two `<Counter />` elements never share state, because they’re different fibers with different `memoizedState` lists.

The full hook catalog

useState — state

The baseline unit of local, per-render state. Internally: reads `memoizedState` off the current hook node; `setState` doesn’t mutate that value in place — it schedules an update and enqueues the

new value, which is only actually written into the fiber during the next render pass. That's why state updates are "asynchronous" relative to the line of code that called `setState`.

```
function Counter() {
  const [count, setCount] = useState(0);

  // Functional update — always correct when the next value
  // depends on the previous one, even under batching
  const increment = () => setCount(c => c + 1);

  return <button onClick={increment}>{count}</button>;
}
```

> **Mistake.** `setCount(count + 1)` called twice in the same handler only increments once, because both calls close over the same stale `count`. `setCount(c => c + 1)` twice correctly increments twice, because each updater receives the result of the previous one in the queue.

Lazy initialization — the second, less-known argument form. Pass a *function* to `useState` instead of a value when the initial value is expensive to compute; React calls it exactly once, on mount, and never again:

```
// Bad — parses the (possibly huge) saved document on every single render,
// then throws the result away every time except the first
const [doc, setDoc] = useState(JSON.parse(localStorage.getItem('draft')));

// Good — the parse only runs once, on mount
const [doc, setDoc] = useState(() =>
        JSON.parse(localStorage.getItem('draft')));
```

Bail-out on identical state. If you call `setState` with a value `Object.is`-equal to the current one, React skips re-rendering that component entirely (though it may still re-render its children if their props changed for other reasons). This is why `setState(x => x)` is a safe no-op, and why toggling a boolean to the value it already holds costs nothing.

useReducer — state

`useState` with an explicit transition function instead of an inline setter. Same linked-list mechanics, but the "how state changes" logic lives in one pure function you can unit-test without

rendering a component.

```
function reducer(state, action) {
  switch (action.type) {
    case 'added': return [...state, action.item];
    case 'removed': return state.filter(i => i.id !== action.id);
    default: throw new Error('Unknown action');
  }
}

const [items, dispatch] = useReducer(reducer, []);
dispatch({ type: 'added', item: newItem });
```

Reach for this over `useState` when: the next state depends on several prior fields at once, several handlers produce overlapping updates, or you want the transition logic testable and colocatable outside the component.

Lazy initialization with an init function works here too, and is the idiomatic way to derive initial reducer state from props without recomputing it on every render:

```
function init(initialCount) {
  return { count: initialCount, history: [] };
}

const [state, dispatch] = useReducer(reducer, initialCount, init);
```

> **Pattern worth knowing.** A reducer that returns a *new object even when nothing logically changed* (e.g. `case 'noop': return { ...state }`) defeats the bail-out optimization above and forces a re-render every dispatch. Return the exact same `state` reference from a branch that shouldn't change anything.

useEffect — synchronization

Runs a callback **after paint**, asynchronously, once the commit is on screen. Its real job — despite how it's commonly taught — is **synchronizing a component with a system outside React's rendering model**: the DOM, a subscription, a timer, a WebSocket, browser storage. Not "run some code after render."

```
useEffect(() => {
  const id = setInterval(() => setTick(t => t + 1), 1000);
  return () => clearInterval(id); // cleanup — runs before the next effect,
        and on unmount
}, []); // deps array — empty means "sync once, on mount"
```

Internals worth knowing: React compares each dependency with `Object.is` against the previous render's array. If *any* entry differs, the previous effect's cleanup runs first, then the new effect body runs. Cleanup and re-run always happen in that order — never both bodies back to back.

> **Mistake.** Omitting a dependency to "stop it re-running so often" instead of fixing the effect to not need that value every time. The dependency array is not a performance knob — it's a correctness contract describing every reactive value the effect body reads. Lying to it produces stale closures (the effect keeps using an old prop or state value forever).

Multiple effects in one component run in declaration order, each one's cleanup-then-setup happening independently — an effect with no changed dependencies simply doesn't re-run, it doesn't get skipped as a block:

```
function Room({ roomId }) {
  useEffect(() => {
    console.log('subscribe to room', roomId);
    return () => console.log('unsubscribe from room', roomId);
  }, [roomId]);

  useEffect(() => {
    document.title = `Room: ${roomId}`;
  }, [roomId]);
  // When roomId changes: unsubscribe(old) → subscribe(new) → set title —
        top-to-bottom, cleanup before setup, per-effect
}
```

The race-condition guard — the single most common real bug in hand-rolled data-fetching effects, worth memorizing even though a library (Part V) should usually own this:

```
useEffect(() => {
  let cancelled = false;
  fetchUser(userId).then((data) => {
    if (!cancelled) setUser(data); // guards against a stale response
  });                              // arriving after a newer request started
  return () => { cancelled = true; };
}, [userId]);
```

Without the `cancelled` flag, rapidly changing `userId` (e.g. clicking through a list fast) can let an older, slower request resolve *after* a newer one and overwrite the correct data with stale data — the classic "why does my UI show the wrong user" bug.

useLayoutEffect — the sync alternative

Identical API to `useEffect`, but it runs **synchronously, immediately after the DOM is mutated and before the browser paints**. Use it when your effect needs to read layout (`getBoundingClientRect`, scroll position) and then synchronously write something back before the user sees a flash of the wrong state.

```
useLayoutEffect(() => {
  const { height } = tooltipRef.current.getBoundingClientRect();
  if (height > window.innerHeight) setPlacement('top');
  // runs before paint — no visible flicker from measuring then adjusting
}, [content]);
```

> **Rule of thumb.** Default to `useEffect`. Reach for `useLayoutEffect` only when you can point at a specific visual flicker it fixes — it blocks paint, so overusing it re-introduces the janky, blocking behavior Fiber was built to avoid.

useMemo — performance

Caches the *result* of an expensive computation across renders, recomputing only when a dependency changes. It is not a correctness tool — a component without it still works, just possibly slower.

```
const sorted = useMemo(
  () => [...items].sort((a, b) => b.score - a.score),
  [items]
);
```

> **Mistake.** Wrapping every derived value in `useMemo` "just in case." The memoization machinery itself has a cost (storing the cached value, comparing deps every render); for cheap computations that cost exceeds the computation you're avoiding. Reserve it for genuinely expensive work (sorting/filtering large arrays, heavy formatting) or for stabilizing a reference passed to `React.memo` /a dependency array.

useCallback — performance

`useMemo` specialized for functions — it exists to give a function a **stable reference** across renders, so it doesn't invalidate a child's `React.memo` or another hook's dependency array.

```
const handleSelect = useCallback((id) => {
  dispatch({ type: 'select', id });
}, [dispatch]); // dispatch is stable, so this never changes

// Only useful because Row is memoized — otherwise Row re-renders
// on every parent render regardless of prop identity
const Row = React.memo(function Row({ onSelect }) { /* ... */ });
```

> **Mistake.** Using `useCallback` on a handler passed to a plain, non-memoized DOM element or component. If nothing downstream cares about reference identity, you've paid the memoization cost for zero benefit.

useRef — escape hatch

A mutable box — `{ current: value }` — that survives across renders **without** triggering a re-render when it changes. Internally it's just another node on the hook linked list whose `memoizedState` holds that object, so the object identity itself never changes between renders.

```
const inputRef = useRef(null);
const renderCount = useRef(0);
renderCount.current++; // mutate freely — this does NOT cause a re-render

return <input ref={inputRef} />;
```

Two distinct uses: holding a DOM node reference (imperative access — focus, measure, scroll), or holding any mutable value you need to persist between renders that shouldn't itself drive UI (a previous value for comparison, an interval ID, a "did I already fetch this" flag).

A common pattern: tracking the previous value of a prop for comparison, since React doesn't give you this natively:

```
function usePrevious(value) {
  const ref = useRef(undefined);
  useEffect(() => {
    ref.current = value; // written AFTER the render that displays `value`,
  });                    // so during render, ref.current still holds the
        previous one
  return ref.current;
}

function PriceTag({ price }) {
  const previousPrice = usePrevious(price);
  const direction = previousPrice === undefined ? null : price >
        previousPrice ? 'up' : 'down';
  return <span className={direction}>${price}</span>;
}
```

> **Mistake.** Reading or writing `ref.current` *during* the render body (not inside an effect or handler) to drive what gets displayed. That's a side effect during render — the same purity violation flagged in Part I — and it breaks under Strict Mode's double-render and concurrent rendering's throwaway renders. Reading a ref during render for a one-time escape hatch (e.g. checking `if (ref.current === null)` to run setup exactly once) is one of the few render-time ref reads React itself considers acceptable — mutating one to store data that feeds back into what's rendered is not.

useContext — state distribution

Subscribes a component to a `Context.Provider` value without threading it through props at every level. Internally, when a provider's value changes, React walks its subtree looking for consumers and schedules them for re-render — every component calling `useContext` on that context re-renders on *any* value change, even if it only cares about part of the value.

```
const ThemeContext = createContext('light');

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>...</div>;
}
```

> **Mistake.** Putting a large, frequently-changing object (like `{ user, theme, cart, notifications }`) in one context. Every consumer re-renders on every field's change. Split contexts by update frequency, or memoize the value object, or move to a selector-based external store (see `useSyncExternalStore`) for anything that updates often.

The other common mistake: an unmemoized value object. Even when the *data* hasn't changed, passing a fresh object literal as the provider's `value` gives every render a new reference, and every consumer re-renders regardless of whether the fields they read actually changed:

```
// Bad — { user, logout } is a brand-new object every render of
        AuthProvider,
// so every consumer re-renders on every AuthProvider render,
        unconditionally
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logout = () => setUser(null);
  return (
    <AuthContext.Provider value={{ user, logout }}>{children}
        </AuthContext.Provider>
  );
}

// Good — same reference across renders unless user actually changes
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const logout = useCallback(() => setUser(null), []);
  const value = useMemo(() => ({ user, logout }), [user, logout]);
  return <AuthContext.Provider value={value}>{children}
        </AuthContext.Provider>;
}
```

useImperativeHandle — escape hatch

Customizes what a parent sees when it attaches a `ref` to your component, instead of exposing the raw DOM node. Used with `forwardRef` (or, in React 19, `ref` as a plain prop).

```
const VideoPlayer = forwardRef((props, ref) => {
  const videoRef = useRef(null);
  useImperativeHandle(ref, () => ({
    play: () => videoRef.current.play(),
    pause: () => videoRef.current.pause(),
  }));
  return <video ref={videoRef} />;
});
```

Rare in practice — most components should communicate through props and callbacks, not imperative handles. Reach for it only when wrapping an inherently imperative API (media playback, canvas, a third-party widget) for a small, deliberate surface.

useId — accessibility / SSR

Generates a stable, unique ID that matches between server-rendered HTML and the client hydration pass — `Math.random()` or a module-level counter would produce different IDs on server vs. client and break hydration (or accessibility attributes that must match, like `aria-describedby`).

```
function Field({ label }) {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>{label}</label>
      <input id={id} />
    </>
  );
}
```

useTransition — concurrent

Marks a state update as **low priority** — React will render it, but will interrupt that render if something more urgent (like a keystroke) comes in, and it won't show a loading fallback for children already on screen.

```
const [isPending, startTransition] = useTransition();
const [query, setQuery] = useState('');
const [results, setResults] = useState([]);

function handleChange(e) {
  setQuery(e.target.value); // urgent — keeps the input responsive
  startTransition(() => {
    setResults(filterHugeList(e.target.value)); // can be interrupted
  });
}
```

This is often the correct replacement for a `useEffect` that debounced or deferred an expensive re-render "to keep the UI smooth" — the transition mechanism does it natively without a timer.

`isPending` is what lets you show a non-blocking "still working" indicator without a spinner that replaces content — the old results stay on screen, dimmed, while the new ones render in the background:

```
return (
  <div style={{ opacity: isPending ? 0.6 : 1 }}>
    <ResultsList results={results} />
  </div>
);
```

> **Mistake.** Wrapping the input's own `setQuery` call in `startTransition`. That defeats the entire point — the input would then lag behind typing, because its update is now competing at low priority with everything else instead of being the one thing kept instantly responsive.

useDeferredValue — concurrent

Same underlying mechanism as `useTransition`, but for a *value* you don't own the setter for (e.g. it comes from props, or from a store). React keeps rendering the old value for expensive consumers while urgent updates happen, then catches the deferred value up in the background.

```
function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  const results = useMemo(
    () => expensiveSearch(deferredQuery),
    [deferredQuery]
  );
  return <List items={results} />;
}
```

Pair it with `React.memo` on the expensive child so React can skip re-rendering that subtree entirely while `deferredQuery` hasn't caught up yet — `useDeferredValue` alone only helps if something downstream is memoized enough to actually be skippable.

```
const List = React.memo(function List({ items }) { /* expensive render */
        });
```

> **useTransition vs. useDeferredValue — the actual difference.** Use `useTransition` when you own the state setter and are choosing to mark *this specific update* as low priority. Use `useDeferredValue` when you don't own the setter — the value arrives via props, context, or a store you don't control — and you want a lagging, low-priority copy of it to render expensive children against.

useSyncExternalStore — the real useEffect alternative

Purpose-built for subscribing to state that lives *outside* React — browser APIs, third-party stores, `localStorage`, WebSocket connections — and it's the one most people reach for `useEffect` + `useState` instead, incorrectly. It's what libraries like Redux and Zustand use under the hood.

```
function useOnlineStatus() {
  return useSyncExternalStore(
    (callback) => { // subscribe
      window.addEventListener('online', callback);
      window.addEventListener('offline', callback);
      return () => {
        window.removeEventListener('online', callback);
        window.removeEventListener('offline', callback);
      };
    },
    () => navigator.onLine, // getSnapshot (client)
    () => true // getServerSnapshot (SSR fallback)
  );
}
```

> **Why this beats useEffect + useState.** The naive version — subscribe in `useEffect`, call `setState` on every event — has a real bug under concurrent rendering: the external store can change value *between* the render phase reading it and the commit phase subscribing to it ("tearing"), producing a UI that's briefly inconsistent with the actual store. `useSyncExternalStore` is specifically designed by the React team to make this tear-free.

A second, equally common use: reading a `localStorage` value reactively across every component that needs it, including reacting to changes made in *other browser tabs* (the native `storage` event only fires cross-tab, which is itself a reason this needs a subscription model, not a one-time read):

```
function useLocalStorage(key, initialValue) {
  const subscribe = (callback) => {
    window.addEventListener('storage', callback);
    return () => window.removeEventListener('storage', callback);
  };
  const getSnapshot = () => localStorage.getItem(key) ?? initialValue;
  return useSyncExternalStore(subscribe, getSnapshot);
}
```

use() — React 19

Reads the value of a Promise or Context, and — unlike other hooks — can be called conditionally. Its main use case is reading a promise inside a component that's wrapped in `<Suspense>`: while the promise is pending, `use()` throws it, Suspense catches that and shows the fallback, and once it resolves the component re-renders with the value.

```
function UserProfile({ userPromise }) {
  const user = use(userPromise); // suspends until resolved
  return <h1>{user.name}</h1>;
}

function Page() {
  return (
    <Suspense fallback={<Spinner />}>
      <UserProfile userPromise={fetchUser()} />
    </Suspense>
  );
}
```

This — combined with a data-fetching library or framework (Next.js, Relay, React Query's `suspense` mode) — is the modern replacement for the "fetch in `useEffect`, track `loading` / `error` / `data` in three `useState` calls" pattern.

Errors work the same way as the pending state: if the promise rejects, `use()` re-throws that rejection during render, and the nearest **error boundary** (not a try/catch — render-phase throws skip past those) catches it and shows its fallback instead of the tree that failed:

```
<ErrorBoundary fallback={<ErrorMessage />}>
  <Suspense fallback={<Spinner />}>
    <UserProfile userPromise={fetchUser()} />
  </Suspense>
</ErrorBoundary>
```

> **Mistake.** Calling `fetchUser()` inside the component body on every render and passing the fresh promise to `use()`. A new promise every render means `use()` suspends forever — it never gets a chance to resolve before being replaced. Create the promise once (in a parent, in a route loader, or via a library that caches it) and pass the *same* promise down until the underlying data actually needs refetching.

Custom hooks — composition

A custom hook is not a special construct — it's a plain function whose name starts with `use` and which calls other hooks. Because of the linked-list mechanics from earlier, every hook it calls gets appended to *the calling component's* fiber, in the position that custom hook occupies in the call order. There's no separate hook state living "inside" the custom hook.

```
function useDebouncedValue(value, delayMs) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);
  return debounced;
}

// Usage — looks like a built-in hook, is just a function
const debouncedQuery = useDebouncedValue(query, 300);
```

Extract a custom hook when the same stateful logic (not just JSX) repeats across components, or when a chunk of a component's hooks form a coherent, independently-testable unit — a form field's validation state, a media query subscription, a fetch-with-cache.

Part III — The useEffect Problem

`useEffect` became the hook everyone reaches for because it's the one taught first and it technically "works" for almost anything — fetching data, responding to prop changes, computing derived values, even reacting to a button click routed through state. That doesn't mean it's the right tool. Every one of those cases either has a purpose-built hook or, more often, doesn't need an effect at all.

The React team's own framing (Dan Abramov's *"You Might Not Need an Effect"*) is worth internalizing as a habit: **before writing `useEffect`, ask what external system you're synchronizing with.** If the honest answer is "nothing, I just wanted code to run after a render," that's a sign the logic belongs somewhere else.

Decision table: what to reach for instead

You're tempted to write an effect for...	Use this instead	Why
Computing one value from another (e.g. `fullName` from `first` + `last`)	`const fullName = first + ' ' + last` — just compute it during render	No external system involved. An effect here means an extra render: one with stale derived state, then one after the effect "catches up."
Resetting all state when a prop (like a user ID) changes	Add `key={userId}` to the component instead	A new key tells React it's a genuinely new component instance — it unmounts the old one (discarding all its state) and mounts a fresh one. No effect, no stale-state window.
Running logic in response to a specific button click / form submit	Put the logic directly in the event handler	The handler already knows exactly what happened and why. Routing it through state + an effect loses that context and fires on renders you didn't intend.
Fetching data on mount / when an ID changes	A data-fetching library (TanStack Query, SWR) or framework loaders; or `use()` + Suspense	Hand-rolled fetch effects are missing race-condition guards, caching, retries, and dedup by default — all of which a real library already solved.
Subscribing to a browser API or external store	`useSyncExternalStore`	Tear-free under concurrent rendering; a hand-rolled subscribe-in-effect is not.
Sending analytics when a value changes	Send it from the event handler that caused the change	An effect fires on every render where the dependency differs — including renders triggered by something unrelated re-mounting the component (e.g. Strict Mode, a key change) — double counting events.
Notifying a parent when internal state changes	Call the parent's callback directly in the handler that changes the state	Same issue — an effect adds a render's worth of lag and can fire for reasons unrelated to a genuine user-driven change.
Adjusting one piece of state when another changes (e.g. clearing `selectedItem` when `items` changes)	Compute it during render, or reset with a conditional inside the render body using a ref guard — not an effect	An effect here always costs one extra "wrong" render first. If you truly can't compute it inline, update state directly during render (a documented, legal exception — see below) rather than scheduling it for after commit.

You're tempted to write an effect for...	Use this instead	Why
Chaining effects (effect A sets state, which triggers effect B, which sets more state)	Rewrite as a single event handler or a single computation that produces every derived value at once	Each link in the chain is a full render+commit cycle. A chain of three effects means three extra renders before the UI is actually correct, and the intermediate states are often visible to the user as a flash.

Before / after: the three most common rewrites

1. Derived state.

```
// Before — an effect just to keep a derived value in sync
const [fullName, setFullName] = useState('');
useEffect(() => {
  setFullName(`${firstName} ${lastName}`);
}, [firstName, lastName]);

// After — no effect, no extra render, always correct
const fullName = `${firstName} ${lastName}`;
```

2. Resetting state when a prop changes.

```
// Before — effect resets fields, but there's a render in between
// where the form still shows the PREVIOUS user's data
useEffect(() => {
  setComment('');
  setDraftSaved(false);
}, [userId]);

// After — a `key` makes this a genuinely new component instance;
// React discards the old one's state instead of you resetting it by hand
<ProfilePage key={userId} userId={userId} />
```

3. Fetching data on mount.

```
// Before — hand-rolled, missing cache, retry, and dedup, and prone
// to the race condition shown earlier in the useEffect section
useEffect(() => {
  setLoading(true);
  fetchUser(userId).then(setUser).finally(() => setLoading(false));
}, [userId]);

// After — TanStack Query owns caching, retries, dedup, and race conditions
const { data: user, isLoading } = useQuery({
  queryKey: ['user', userId],
  queryFn: () => fetchUser(userId),
});
```

> **The legal exception: adjusting state during render.** React explicitly permits calling `setState` directly in the render body (not inside an effect) *while a component is rendering*, guarded by a check against the previous props, specifically for the "reset a piece of state when a prop changes" case when a `key` isn't practical:
>
> ```
> function List({ items, selectedId }) {
>   const [prevItems, setPrevItems] = useState(items);
>   const [selection, setSelection] = useState(selectedId);
>   if (items !== prevItems) {
>     setPrevItems(items);
>     setSelection(selectedId); // adjusts selection in the SAME
>          render, no extra commit
>   }
>   // ...
> }
> ```
>
> This looks alarming ("mutating state during render!") but React specifically detects this pattern (a conditional `setState` call gated on a changed value) and re-renders immediately with the new state before committing anything to the DOM — so the user never sees the stale intermediate frame an effect would produce. Reach for this only when a `key` reset is impractical (e.g. you need to keep some fields and reset others) — it is uncommon, not a first choice.

When useEffect is actually the correct tool

It's not that effects are bad — they're the correct hook for genuinely synchronizing React with something it doesn't manage:

- Directly manipulating a non-React widget you've mounted into a DOM node (a map library, a chart, a rich text editor).
- Setting up a subscription/timer/interval whose lifecycle should track the component's mount/unmount (though prefer `useSyncExternalStore` if you're just reading a value from it).
- Synchronizing document-level state — `document.title`, focus management, scroll restoration.
- Logging/instrumentation that's genuinely meant to fire "whenever this component is on screen with these props," not tied to one user action.

> **The one-line test.** If you can point at the non-React system your effect is keeping in sync with the DOM you just rendered, it's a real effect. If you're describing "something that should happen after this state changes," it's derived logic or event-handler logic wearing an effect as a costume.

Part IV — Production Practice

Choosing where state lives

State shape	Put it in
Used by one component and its direct children	`useState` / `useReducer` in that component
Shared across a subtree, changes rarely (theme, auth user, locale)	`useContext` + `useReducer`
Shared across the app, changes often, needs selectors to avoid over-rendering	An external store (Zustand, Redux) via `useSyncExternalStore`-based bindings
Anything that originated on a server — API responses, DB records	A server-state library (TanStack Query, SWR, or your framework's data layer) — never plain `useState`
Form field values and validation	A form library (React Hook Form, Formik) for anything beyond 2–3 fields
URL-shareable state (filters, active tab, pagination)	The URL (search params), not component state

> **Production pattern.** The most common real-world mistake isn't a hook misuse — it's treating server data as component state. The instant you put fetched data in `useState`, you've taken on cache invalidation, refetching, race conditions, and loading/error state tracking yourself. A dedicated server-state library gives you all of that for the cost of one dependency.

Performance checklist (in the order to actually check them)

1. **Profile first.** Use React DevTools Profiler before optimizing anything — memoizing a component that wasn't slow adds complexity for zero benefit.
2. **Fix the render trigger, not the render cost.** A component re-rendering 200 times because its parent's state changes is a structural problem — move state down, split components, or use children-as-props to avoid the parent's re-render creating new child element instances.
3. **Memoize expensive computation** with `useMemo`, not every computation.
4. **Stabilize references** passed to memoized children with `useCallback` / `useMemo`, and only where the child is actually wrapped in `React.memo`.
5. **Virtualize long lists** (react-window / TanStack Virtual) instead of rendering thousands of DOM nodes.
6. **Code-split** with `React.lazy` + `Suspense` at route boundaries before micro-optimizing render cost.

Error boundaries: containing failure to a subtree

A thrown error during render, with no error boundary above it, unmounts the *entire* React tree — one broken widget takes down the whole page. An error boundary is a component (must be a class — there's no hook equivalent, since it needs `getDerivedStateFromError` / `componentDidCatch` lifecycle methods) that catches errors thrown by its children during rendering and shows a fallback instead.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  componentDidCatch(error, info) {
    logErrorToService(error, info); // report it — the boundary doesn't do
         this for you
  }
  render() {
    if (this.state.hasError) return this.props.fallback;
    return this.props.children;
  }
}

// Usage — scope it around the riskiest, most independent subtree,
// not just once around the whole app
<ErrorBoundary fallback={<WidgetCrashed />}>
  <FlakyThirdPartyWidget />
</ErrorBoundary>
```

> **What it does not catch:** errors inside event handlers (those are regular JS exceptions — use try/catch), errors in asynchronous code outside render (a `setTimeout` callback, a raw `.then()` not read via `use()`), and errors during server-side rendering. It only catches errors thrown *during the render phase* of its child tree — which lines up exactly with why `use()`'s promise-rejection re-throw in Part II is designed to surface there.

Code-splitting in practice

`React.lazy` defers loading a component's code until it's actually rendered, paired with `Suspense` to show a fallback while the chunk downloads:

```
const SettingsPage = React.lazy(() => import('./SettingsPage'));

<Suspense fallback={<PageSkeleton />}>
  <Routes>
    <Route path="/settings" element={<SettingsPage />} />
  </Routes>
</Suspense>
```

The highest-leverage place to do this is at route boundaries (Part V's React Router section shows the equivalent done through the router itself) — a user visiting `/` shouldn't download the code for `/settings` or `/admin` until they navigate there. Splitting inside a single visible page (e.g. lazy-loading one modal's code) has real but much smaller payoff, since it's a small fraction of the page's total bundle.

Part V — The Ecosystem

Beyond core React: the libraries every production app reaches for

React itself ships none of these: no router, no global store, no HTTP client, no form library, no test runner opinion. That's deliberate — React is a rendering library, not a framework — but it means every real app converges on roughly the same short list of companions. Here's each one, what problem it actually solves, and where it fits against what's already in this guide.

React Router — client-side routing

React has no concept of a "page." Without a router, every URL change is either a full page reload (losing all client state) or something you'd have to hand-roll with `window.history` and a big conditional. React Router owns that: it maps URL patterns to components, and (since v6.4's data APIs) can load a route's data *before* it renders instead of rendering first and fetching in an effect.

```
const router = createBrowserRouter([
  {
    path: '/products/:id',
    element: <ProductPage />,
    loader: ({ params }) => fetchProduct(params.id), // data ready before
        render
  },
]);

function ProductPage() {
  const product = useLoaderData(); // no loading state to manage here
  return <h1>{product.name}</h1>;
}
```

> **How this connects back.** A route `loader` is the router-level version of the same idea as `use()` + Suspense from Part II: fetch before you render, instead of rendering empty and patching in data via a `useEffect`. "URL-shareable state (filters, tab, page) belongs in the URL" from Part IV is what React Router's `useSearchParams` gives you a typed-ish hook for.

Nested routes and shared layouts. Routes nest to match how UI actually nests — a dashboard layout with a persistent sidebar, and a changing page inside it, is one parent route with children, not a full-page component re-rendering the sidebar on every navigation:

```
const router = createBrowserRouter([
  {
    path: '/app',
    element: <DashboardLayout />,       // renders sidebar + <Outlet />, once
    children: [
      { path: 'overview', element: <Overview /> },   // renders into <Outlet />
      { path: 'billing', element: <Billing /> },
    ],
  },
]);

function DashboardLayout() {
  return (
    <div className="dashboard">
      <Sidebar />
      <Outlet /> {/* the matched child route renders here */}
    </div>
  );
}
```

Protected routes — gating a subtree behind auth is a plain component, not a router feature; the router just needs somewhere to redirect from:

```
function RequireAuth({ children }) {
  const { user } = useAuth(); // e.g. a Zustand or Context-based auth store
  const location = useLocation();
  if (!user) {
    // remember where they were headed so login can send them back
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
  return children;
}

// Usage
{ path: '/app', element: <RequireAuth><DashboardLayout /></RequireAuth>,
          children: [...] }
```

Route-level code splitting and error elements — combining Part IV's `React.lazy` with a per-route error boundary, so one broken page doesn't take down the whole app's routing:

```
const Billing = React.lazy(() => import('./Billing'));

{
  path: 'billing',
  element: <Suspense fallback={<PageSkeleton />}><Billing /></Suspense>,
  errorElement: <RouteErrorFallback />, // catches loader errors AND render
        errors for this route
}
```

Programmatic navigation and reading params — the two hooks you'll reach for constantly outside of loaders:

```
function ProductCard({ product }) {
  const navigate = useNavigate();
  return <button onClick={() =>
        navigate(`/products/${product.id}`)}>View</button>;
}

function ProductPage() {
  const { id } = useParams(); // typed as string — parse before using as a
        number
  const [searchParams, setSearchParams] = useSearchParams();
  const sort = searchParams.get('sort') ?? 'newest';
  // ...
}
```

Redux Toolkit — global client state, with rules

Redux Toolkit (RTK) is the current, official way to write Redux — the older hand-written-action-types-and-switch-reducers Redux is effectively deprecated. It solves a specific problem: when app-wide client state (not server data — see the state-placement table in Part IV) is complex enough, or touched by enough unrelated components, that `useContext`'s "every consumer re-renders on any change" becomes a real cost, and you want time-travel debugging, middleware, and a strict, traceable "how did the state change" trail.

```
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    added(state, action) {
      state.items.push(action.payload); // looks like a mutation —
    },                                  // Immer drafts it into an
        immutable update
    removed(state, action) {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
  },
});

const store = configureStore({ reducer: { cart: cartSlice.reducer } });

// in a component — only re-renders when the selected slice changes
const items = useSelector((state) => state.cart.items);
const dispatch = useDispatch();
dispatch(cartSlice.actions.added(newItem));
```

> **Why RTK over plain Context.** `useSelector` is built on the same `useSyncExternalStore`-style subscription model from Part II: components subscribe to a *slice* of state and only re-render when that slice's selector output changes — Context has no equivalent, it re-renders every consumer on any value change. RTK also bundles Immer (so reducers can read as mutations while staying pure) and RTK Query (a built-in data-fetching layer that overlaps with TanStack Query — pick one, not both).

> **Mistake.** Reaching for Redux Toolkit by default. If nothing you're storing is genuinely cross-cutting and frequently updated by many unrelated parts of the tree, `useContext` + `useReducer` or a small Zustand store gets you 90% of the benefit with far less boilerplate and setup.

Async logic: `createAsyncThunk`. Plain reducers must be synchronous and pure — they can't call an API. A thunk wraps an async function and automatically dispatches `pending` / `fulfilled` / `rejected` actions around it, which the slice can handle directly:

```
const fetchOrders = createAsyncThunk('orders/fetch', async (userId) => {
  const response = await api.get(`/users/${userId}/orders`);
  return response.data;
});

const ordersSlice = createSlice({
  name: 'orders',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchOrders.pending, (state) => { state.status = 'loading';
        })
      .addCase(fetchOrders.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items = action.payload;
      })
      .addCase(fetchOrders.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message;
      });
  },
});

// in a component
dispatch(fetchOrders(userId));
```

RTK Query — Redux's built-in answer to TanStack Query. If you're already committed to Redux Toolkit for client state, RTK Query gives you the caching/refetching/dedup layer from Parts III–IV without a second library, generating hooks from an API definition:

```
const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (builder) => ({
    getOrders: builder.query({ query: (userId) => `/users/${userId}/orders`
        }),
    addOrder: builder.mutation({
      query: (order) => ({ url: '/orders', method: 'POST', body: order }),
      invalidatesTags: ['Orders'], // triggers a refetch of getOrders after
         this mutation
    }),
  }),
});

// auto-generated hook — behaves like useQuery from TanStack Query
const { data: orders, isLoading } = api.useGetOrdersQuery(userId);
```

> **Pick one, not both.** RTK Query and TanStack Query solve the same problem. If you're not otherwise using Redux for client state, TanStack Query alone is simpler to adopt. If you already have a Redux store for other reasons, RTK Query avoids running two separate caching systems side by side.

Axios vs. fetch — the HTTP client question

`fetch` is built into the browser and covers the basics, but it makes you hand-roll things Axios gives you by default: automatic JSON parsing (fetch requires an explicit `.json()` call and doesn't reject on 4xx/5xx status codes — you have to check `response.ok` yourself), request/response interceptors, and built-in request cancellation and timeout support.

```
const api = axios.create({
  baseURL: '/api',
  timeout: 10000,
});

// interceptor — attach auth once, everywhere, instead of per-call
api.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config;
});

// rejects on 4xx/5xx automatically — fetch does not
const { data } = await api.get('/products/42');
```

> **Where it actually belongs.** Axios (or fetch) is the transport — the function that goes and gets bytes. It is not a replacement for TanStack Query/SWR from Parts III–IV, which handle caching, refetching, and race conditions *on top of* whatever transport you use. A very common production shape: TanStack Query's `queryFn` calls an Axios instance.

The token-refresh interceptor — the single most-copied piece of production Axios setup, and worth understanding rather than just pasting: on a 401, silently refresh the access token once, retry the original request, and queue up any other requests that failed while the refresh was in flight so they don't each trigger their own refresh:

```
let isRefreshing = false;
let pendingQueue = [];

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const { config, response } = error;
    if (response?.status !== 401 || config._retried) {
      return Promise.reject(error); // not a token problem, or already
         retried once
    }
    if (isRefreshing) {
      // another request already triggered a refresh — wait for it instead
         of refreshing again
      return new Promise((resolve, reject) => {
        pendingQueue.push({ resolve, reject, config });
      });
    }
    config._retried = true;
    isRefreshing = true;
    try {
      const { data } = await axios.post('/auth/refresh', {}, {
         withCredentials: true });
      setToken(data.accessToken);
      pendingQueue.forEach(({ resolve, config: c }) => resolve(api(c)));
      pendingQueue = [];
      return api(config); // retry the original request with the new token
    } catch (refreshError) {
      pendingQueue.forEach(({ reject }) => reject(refreshError));
      pendingQueue = [];
      logout();
      return Promise.reject(refreshError);
    } finally {
      isRefreshing = false;
    }
  }
);
```

Cancellation with `AbortController` — the fix for the same stale-response race condition the `useEffect` section flagged earlier, done at the transport layer instead of with a boolean flag:

```
useEffect(() => {
  const controller = new AbortController();
  api.get(`/users/${userId}`, { signal: controller.signal })
    .then((res) => setUser(res.data))
    .catch((err) => { if (!axios.isCancel(err)) throw err; });
  return () => controller.abort(); // cancels the in-flight request
        outright, not just its effect on state
}, [userId]);
```

Zustand — state management without the ceremony

Sits between Context and Redux Toolkit: a global store with no boilerplate (no actions, no providers, no reducers required), built directly on the subscribe-and-select model, so — like `useSelector` — components only re-render when the specific slice they read changes.

```
const useCartStore = create((set) => ({
  items: [],
  add: (item) => set((state) => ({ items: [...state.items, item] })),
  remove: (id) => set((state) => ({
    items: state.items.filter(i => i.id !== id)
  })),
}));

// no Provider needed — and only re-renders when `items` changes,
// not on every store update, because of the selector
function CartBadge() {
  const count = useCartStore((state) => state.items.length);
  return <span>{count}</span>;
}
```

Reach for Zustand over Redux Toolkit when you want the same “any component, any depth, no prop drilling, selective re-renders” benefit without adopting Redux’s stricter action/reducer conventions — most greenfield apps that would have defaulted to Redux in 2019 default to Zustand now.

Middleware: `persist` and `devtools`. Zustand's middleware system is where it picks up the two things you'd otherwise lose by skipping Redux — Redux DevTools time-travel debugging, and automatic `localStorage` persistence:

```
const useCartStore = create(
  devtools(
    persist(
      (set) => ({
        items: [],
        add: (item) => set((state) => ({ items: [...state.items, item] })),
      }),
      { name: 'cart-storage' } // localStorage key — rehydrates
         automatically on load
    )
  )
);
```

Slices, for a store that's outgrown one file. Once a store covers several unrelated concerns (cart, auth, UI preferences), split it into slices and compose them — the same shape RTK's multiple `createSlice` calls give you, without a separate library:

```
const createCartSlice = (set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
});

const createAuthSlice = (set) => ({
  user: null,
  login: (user) => set({ user }),
});

const useStore = create((...a) => ({
  ...createCartSlice(...a),
  ...createAuthSlice(...a),
}));
```

Two-way binding: what it means, and why React doesn't really have it

Angular (`ngModel`) and Vue (`v-model`) ship **two-way binding** as a language feature: point a directive at a variable, and the framework wires both directions for you — type in the input and the variable updates, change the variable and the input updates — without you writing either wire yourself.

React deliberately has no equivalent directive. Its core model is **one-way data flow**: state flows down as props, events flow up as callbacks, and *you* are the one who connects them. Everything that looks like two-way binding in React — including what `useForm` does — is one of a few explicit patterns built on top of that one-way flow, not a framework feature.

Controlled components — two-way binding, hand-wired. This is the pattern every `useState` -backed input uses, and it's worth seeing as two separate, explicit wires instead of one magic mechanism:

```
const [name, setName] = useState('');

<input
  value={name}                          // wire 1: state → input (React sets
        the DOM value every render)
  onChange={(e) => setName(e.target.value)} // wire 2: input → state (you
        read the DOM value back out)
/>
```

Together, those two lines behave like two-way binding — but you wrote both directions yourself, which is exactly why React can do things Angular/Vue's automatic binding can't easily express inline: transform the value on the way in (`setName(e.target.value.toUpperCase())`), reject a keystroke (`if (/^\d*$/.test(v)) setName(v)` for a digits-only field), or derive a second piece of state from the same event. The cost: React re-renders the component on every keystroke, because `setName` is a real state update.

Uncontrolled components — no binding at all. The DOM owns the value; React never sees each keystroke, so there's no per-keystroke re-render. You "pull" the value out only when you actually need it:

```
const inputRef = useRef(null);

<input ref={inputRef} defaultValue="" />

// read it imperatively, only when it matters — e.g. on submit
function handleSubmit() {
  console.log(inputRef.current.value);
}
```

A custom "two-way binding" hook — the thing tutorials titled "two-way binding in React" usually show. It's sugar over the exact same controlled pattern above, spread onto the input instead of written out by hand — not a new mechanism, just less repetition across many fields:

```
function useBinding(initialValue) {
  const [value, setValue] = useState(initialValue);
  return { value, onChange: (e) => setValue(e.target.value) };
}

function ProfileForm() {
  const name = useBinding('');
  const email = useBinding('');
  return (
    <>
      <input {...name} />
      <input {...email} />
    </>
  );
}
```

Where `useForm` actually fits. React Hook Form's `register` is neither of the above by default — it wires an **uncontrolled** input (like the `useRef` example: the DOM owns each keystroke, no re-render per character), but layers a subscription system on top so validation state (`errors` , `isSubmitting` , `isDirty`) still updates reactively, re-rendering only the pieces of the form that need to show something:

```
const { register, formState: { errors } } = useForm();

<input {...register('email', { required: true })} />
// register() returns { name, ref, onChange, onBlur } — RHF reads the value
         via
// the ref when it needs it (on blur, on submit, on validation), not on
         every keystroke
```

For a third-party component that *only* accepts a controlled `value` / `onChange` pair and exposes no usable `ref` (a date picker, a custom `<Select>`), RHF's `Controller` switches that one field back to the classic controlled pattern for you, scoped so only that field's wrapper re-renders:

```
<Controller
  name="startDate"
  control={control}
  render={({ field }) => <DatePicker selected={field.value} onChange=
         {field.onChange} />}
/>
```

Approach	Who owns the value	Re-renders per keystroke	Best for
Controlled (`useState` + `onChange`)	React state	Yes, that component	Simple forms, live validation/formatting/masking as the user types, or other UI that must react to every keystroke
Uncontrolled (`useRef`)	The DOM	No	One-off inputs, file inputs, wrapping non-React widgets
React Hook Form `register`	The DOM (uncontrolled under the hood)	No — only fields whose validation state actually changed	Any form beyond 2–3 fields, especially with per-field validation
React Hook Form `Controller`	React state (RHF re-implements controlled binding for that one field)	Only that field's wrapper	Wiring a controlled-only third-party input component

> **The takeaway.** "Does React have two-way binding?" — no, by design, and `useForm` doesn't add it back either. It gives you validation and form state management on top of *uncontrolled* inputs by default, which is precisely how it avoids the re-render-per-keystroke cost that a fully controlled multi-field form pays.

React Hook Form — forms without a re-render per keystroke

A plain `useState`-per-field form re-renders the entire form component on every keystroke in every field. React Hook Form keeps field values in uncontrolled refs internally and only re-renders when validation state actually needs to show something — the input the user is typing into re-renders itself, not the surrounding form.

```
const { register, handleSubmit, formState: { errors } } = useForm();

const onSubmit = (data) => createAccount(data);

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register('email', { required: 'Email is required' })} />
    {errors.email && <span>{errors.email.message}</span>}
    <button type="submit">Create account</button>
  </form>
);
```

> **Mistake.** Building a multi-field form entirely from scratch with `useState` per field once it grows past 2–3 fields with cross-field validation. It's a fine learning exercise once — in production code, that's exactly the "form field values" row from the state-placement table in Part IV.

Schema-based validation with Zod, instead of scattered `required` / `pattern` rules. For anything beyond trivial validation, define the shape once as a schema and let a resolver translate it into React Hook Form's error format — the same schema can also validate the payload server-side if your backend is also TypeScript/JS:

```
const schema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(8, 'At least 8 characters'),
  confirmPassword: z.string(),
}).refine((data) => data.password === data.confirmPassword, {
  message: 'Passwords must match',
  path: ['confirmPassword'],
});

const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema),
});
```

`Controller`, for wiring up a component that isn't a plain `<input>` — a date picker, a rich select, a custom-styled toggle — anything that doesn't expose a native `ref` / `onChange` pair `register` can attach to directly:

```
<Controller
  name="startDate"
  control={control}
  render={({ field }) => <DatePicker selected={field.value} onChange=
         {field.onChange} />}
/>
```

Testing Library + Vitest — testing behavior, not internals

React Testing Library's entire philosophy is one rule: **test your components the way a user would use them** — query by visible text/role/label, click things, assert on what's on screen. It deliberately makes it awkward to reach into a component's internal state or hook values, because tests coupled to internals break on every refactor even when behavior didn't change.

```
import { render, screen, fireEvent } from '@testing-library/react';

test('increments the counter on click', () => {
  render(<Counter />);
  const button = screen.getByRole('button', { name: /count: 0/i });
  fireEvent.click(button);
  expect(screen.getByRole('button', { name: /count: 1/i
        })).toBeInTheDocument();
});
```

Vitest (or Jest) is the runner underneath — it provides `test` / `expect` /mocking; Testing Library is the rendering + querying layer on top, specific to component UIs.

Mocking the network with MSW (Mock Service Worker), instead of mocking `fetch` /Axios directly. MSW intercepts requests at the network level, so the component under test runs its real data-fetching code unmodified — you're testing the actual integration between component and TanStack Query/Axios, not a stubbed-out function:

```
const server = setupServer(
  http.get('/api/users/:id', ({ params }) => {
    return HttpResponse.json({ id: params.id, name: 'Ada Lovelace' });
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

test('shows the fetched user name', async () => {
  render(<UserProfile userId="1" />, { wrapper: QueryClientProviderWrapper
        });
  expect(await screen.findByText('Ada Lovelace')).toBeInTheDocument(); //
        findBy* waits for async updates
});
```

Integration tests over routes, wrapping a component in a real (in-memory) router instead of just the component in isolation — this is what actually catches bugs in `RequireAuth`, redirects, and `useParams` usage:

```
test('redirects unauthenticated users to /login', () => {
  const router = createMemoryRouter(routes, { initialEntries:
        ['/app/billing'] });
  render(<RouterProvider router={router} />);
  expect(screen.getByRole('heading', { name: /log in/i
        })).toBeInTheDocument();
});
```

> **The line to hold.** `getByTestId` is the escape hatch, not the default — reach for it only when no accessible role/label/text exists to query by, since a test built on `data-testid` doesn't verify the UI is actually usable (by a screen reader, by a real user scanning for a labeled button) the way `getByRole` does.

The stack at a glance

Need	Reach for	Covered in
Multiple pages / URL-driven views	React Router	Part V — React Router
Complex, cross-cutting client state with strict conventions	Redux Toolkit	Part V — Redux Toolkit
Global client state, minimal ceremony	Zustand	Part V — Zustand
Talking to an HTTP API	Axios (or fetch)	Part V — Axios vs. fetch
Caching/refetching/deduping server data	TanStack Query / SWR	Part III & IV
Forms beyond 2–3 fields	React Hook Form	Part V — React Hook Form
Verifying behavior, not implementation	Vitest + React Testing Library	Part V — Testing
Build tooling / dev server	Vite	—
Type safety across props, state, API responses	TypeScript	—

Part VI — Build

Projects to build, in order

Each project is scoped to force you to actually use the concepts above, not just read about them. Do them roughly in order — later ones assume the earlier patterns are comfortable.

Beginner — get the fundamentals under your fingers

01. Multi-step form wizard A signup flow with 3–4 steps, back/next navigation, and validation that persists across steps. *Skills: useReducer for step + form state, controlled inputs, derived validation (no effects).*

02. Dark-mode-aware dashboard shell Sidebar + header layout with a theme toggle that persists to `localStorage` and respects system preference. *Skills: useContext, useSyncExternalStore (for the media-query + storage sync), key-based resets.*

03. Debounced search box against a public API Type-ahead search (try a free API like a books or countries API) with loading/empty/error states. *Skills: custom useDebouncedValue hook, correct effect cleanup to cancel stale requests (or swap to TanStack Query and compare).*

Intermediate — where architecture starts to matter

04. Kanban board (Trello-style) Drag-and-drop columns and cards, with reordering, editing, and undo. *Skills: correct keys under reordering, useReducer for complex nested state, useRef for drag coordinates, React.memo tuning with the Profiler.*

05. Real-time chat UI over WebSocket Connect to a WebSocket echo server or a small backend you write; show connection status, message history, typing indicator. *Skills: useSyncExternalStore for connection state, proper effect cleanup for the socket lifecycle, virtualized message list.*

06. Data table with server-driven filters Sortable, filterable, paginated table where filters live in the URL and data comes from a real API. *Skills: TanStack Query for server state, URL as state, useDeferredValue on the filter input for large datasets.*

Advanced — production-shaped systems

07. Collaborative note editor with optimistic updates Rich-text notes that save to a backend optimistically, roll back on failure, and show sync status per note. *Skills: TanStack Query mutations + optimistic updates, useTransition for non-blocking saves, useImperativeHandle wrapping a text-editor library.*

08. Command palette (Cmd-K style) Fuzzy-searchable global command menu with keyboard navigation, portals, and focus trapping. *Skills: useId for a11y wiring, useLayoutEffect for measuring/positioning, useDeferredValue for the fuzzy search, portals.*

09. Streaming AI chat interface A chat UI that streams tokens from an LLM API as they arrive, with stop/regenerate and markdown rendering mid-stream. *Skills: use() with Suspense for the initial load, ReadableStream consumption inside an effect (a legitimate external-system case), careful re-render batching for token-by-token updates.*

10. Mini design-system + component playground Build 10–12 primitives (Button, Modal, Tooltip, Tabs, Toast) from scratch, each with a docs page showing variants and a11y states. *Skills: compound components, forwardRef/useImperativeHandle done right, context for compound state (Tabs), portals for Modal/Toast, focus management.*

Reference points worth reading alongside this guide: the React team's "You Might Not Need an Effect" in the official docs, and the "React Behind the Scenes" series on Medium covering Virtual DOM, reconciliation, and Fiber in more depth. This guide condenses and reorganizes that material around production decisions rather than internals trivia.